Name -Anish

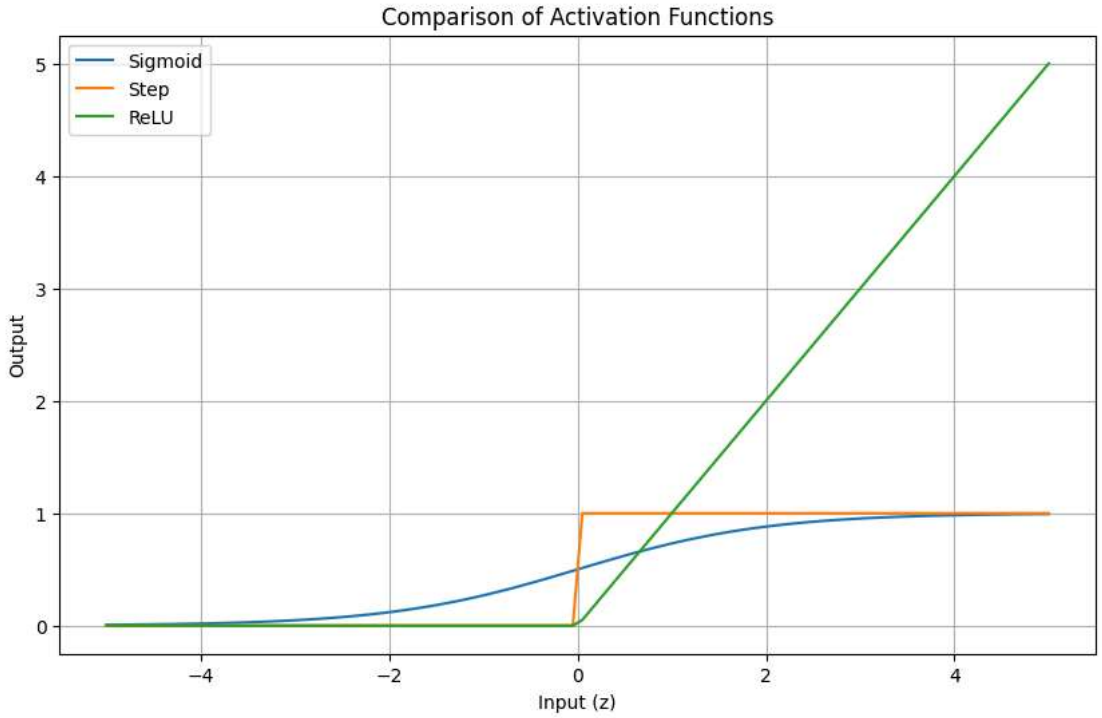
Roll No. 1/23/SET/BCS/419

6 AIMLB1

Perform "Step" , "Sigmoid" , "Relu" activation functions

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def art_neuron(w, x, b, activation='step'):
  z = np.dot(w, x) + b
  if activation == 'step':
    return 1 if z >= 0 else 0
  elif activation == 'sigmoid':
    return 1 / (1 + np.exp(-z))
  elif activation == 'relu':
    return np.maximum(0, z)

x = np.array([0.8, 0.3])
w = np.array([0.2, 0.1])
b = 0.5

def sigmoid_direct(z):
    return 1 / (1 + np.exp(-z))

def step_direct(z):
    return np.where(z >= 0, 1, 0)

def relu_direct(z):
    return np.maximum(0, z)

print("\n\nSigmoid Output : ", art_neuron(w, x, b, 'sigmoid'))
print("Step Output : ", art_neuron(w, x, b, 'step'))
print("Relu Output : ", art_neuron(w, x, b, 'relu'))

z_values = np.linspace(-5, 5, 100)

sigmoid_outputs_direct = sigmoid_direct(z_values)
step_outputs_direct = step_direct(z_values)
relu_outputs_direct = relu_direct(z_values)

plt.figure(figsize=(10, 6))
sns.lineplot(x=z_values, y=sigmoid_outputs_direct, label='Sigmoid')
sns.lineplot(x=z_values, y=step_outputs_direct, label='Step')
sns.lineplot(x=z_values, y=relu_outputs_direct, label='ReLU')

plt.title('Comparison of Activation Functions')
plt.xlabel('Input (z)')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

```
Sigmoid Output :  0.6659669267518202
Step Output :  1
Relu Output :  0.6900000000000001
```



MC Culloch pitts "AND GATE" & "OR GATE" & "NOT GATE" & "NAND GATE" implementation

For 2 inputs

```
def mp_and(x1, x2):
    w1, w2 = 1, 1
    theta = 2
    z = x1*w1 + x2*w2
    return 1 if z >= theta else 0

print(" AND GATE using Mc Culloch Pitts Neuron\n")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        print({x1}, {x2},"-", mp_and(x1, x2))
```

```
 AND GATE using Mc Culloch Pitts Neuron

{0} {0} - 0
{0} {1} - 0
{1} {0} - 0
{1} {1} - 1
```

```
def mp_or(x1, x2):
    w1, w2 = 1, 1
    theta = 1
    z = x1*w1 + x2*w2
    return 1 if z >= theta else 0

print(" OR GATE using Mc Culloch Pitts Neuron\n")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        print({x1}, {x2},"-", mp_or(x1, x2))
```

```
 OR GATE using Mc Culloch Pitts Neuron

{0} {0} - 0
{0} {1} - 1
{1} {0} - 1
{1} {1} - 1
```

```
def mp_not(x):
    w = -1
    theta = 0
    z = x * w
    return 1 if z >= theta else 0

print(" NOT GATE using Mc Culloch Pitts Neuron\n")
for x in [0, 1]:
    print({x}, "-", mp_not(x))
```

```
 NOT GATE using Mc Culloch Pitts Neuron

{0} - 1
{1} - 0
```

```
def mp_nand(x1, x2):
    w1, w2 = -1, -1
    theta = -1
    z = x1*w1 + x2*w2
    return 1 if z >= theta else 0

print(" NAND GATE using Mc Culloch Pitts Neuron\n")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        print({x1},{x2}, "-", mp_nand(x1, x2))
```

```
 NAND GATE using Mc Culloch Pitts Neuron

{0} {0} - 1
{0} {1} - 1
{1} {0} - 1
{1} {1} - 0
```

For any input

```
import numpy as np
import itertools

def mc_pitts(w, x, theta):
  return 1 if np.dot(w, x) >= theta else 0

print("AND GATE using Mc Culloch Pitts Neuron for any number of inputs")

num_inputs = int(input("Enter the number of inputs for the AND gate: "))

w = np.array([1] * num_inputs)

theta = num_inputs

print(f"\nTesting AND Gate with {num_inputs} inputs:")
print(f"Weights: {w}, Theta: {theta}")
for inputs_tuple in itertools.product([0, 1], repeat=num_inputs):
    x = np.array(inputs_tuple)
    output = mc_pitts(w, x, theta)
    print(f"Inputs: {inputs_tuple} -> Output: {output}")
```

```
AND GATE using Mc Culloch Pitts Neuron for any number of inputs
Enter the number of inputs for the AND gate: 4

Testing AND Gate with 4 inputs:
Weights: [1 1 1 1], Theta: 4
Inputs: (0, 0, 0, 0) -> Output: 0
Inputs: (0, 0, 0, 1) -> Output: 0
Inputs: (0, 0, 1, 0) -> Output: 0
Inputs: (0, 0, 1, 1) -> Output: 0
Inputs: (0, 1, 0, 0) -> Output: 0
Inputs: (0, 1, 0, 1) -> Output: 0
Inputs: (0, 1, 1, 0) -> Output: 0
Inputs: (0, 1, 1, 1) -> Output: 0
Inputs: (1, 0, 0, 0) -> Output: 0
Inputs: (1, 0, 0, 1) -> Output: 0
Inputs: (1, 0, 1, 0) -> Output: 0
Inputs: (1, 0, 1, 1) -> Output: 0
Inputs: (1, 1, 0, 0) -> Output: 0
```

```
Inputs: (1, 1, 0, 1) -> Output: 0
Inputs: (1, 1, 1, 0) -> Output: 0
Inputs: (1, 1, 1, 1) -> Output: 1
```

```
import numpy as np
import itertools

def mc_pitts_or(inputs, weights, theta):
  z = np.dot(inputs, weights)

  return 1 if z >= theta else 0

print("OR GATE using Mc Culloch Pitts Neuron for any input\n")

num_inputs = int(input("Enter the number of inputs for the OR gate: "))

weights = np.array([1] * num_inputs)

theta = 1

print(f"\nTesting OR Gate with {num_inputs} inputs:")
print(f"Weights: {weights}, Theta: {theta}")

for inputs_tuple in itertools.product([0, 1], repeat=num_inputs):
    inputs_array = np.array(inputs_tuple)
    output = mc_pitts_or(inputs_array, weights, theta)
    print(f"Inputs: {inputs_tuple} -> Output: {output}")
```

```
OR GATE using Mc Culloch Pitts Neuron for any input

Enter the number of inputs for the OR gate: 4

Testing OR Gate with 4 inputs:
Weights: [1 1 1 1], Theta: 1
Inputs: (0, 0, 0, 0) -> Output: 0
Inputs: (0, 0, 0, 1) -> Output: 1
Inputs: (0, 0, 1, 0) -> Output: 1
Inputs: (0, 0, 1, 1) -> Output: 1
Inputs: (0, 1, 0, 0) -> Output: 1
Inputs: (0, 1, 0, 1) -> Output: 1
Inputs: (0, 1, 1, 0) -> Output: 1
Inputs: (0, 1, 1, 1) -> Output: 1
Inputs: (1, 0, 0, 0) -> Output: 1
Inputs: (1, 0, 0, 1) -> Output: 1
Inputs: (1, 0, 1, 0) -> Output: 1
Inputs: (1, 0, 1, 1) -> Output: 1
Inputs: (1, 1, 0, 0) -> Output: 1
Inputs: (1, 1, 0, 1) -> Output: 1
Inputs: (1, 1, 1, 0) -> Output: 1
Inputs: (1, 1, 1, 1) -> Output: 1
```

```
import numpy as np
import itertools

def mc_pitts_nand(inputs, weights, theta):
    z = np.dot(inputs, weights)
    return 1 if z >= theta else 0

print("NAND GATE using Mc Culloch Pitts Neuron for any input\n")

num_inputs = int(input("Enter the number of inputs for the NAND gate: "))

weights = np.array([-1] * num_inputs)

theta = -(num_inputs - 1)

print(f"\nTesting NAND Gate with {num_inputs} inputs:")
print(f"Weights: {weights}, Theta: {theta}")

for inputs_tuple in itertools.product([0, 1], repeat=num_inputs):
    inputs_array = np.array(inputs_tuple)
    output = mc_pitts_nand(inputs_array, weights, theta)
    print(f"Inputs: {inputs_tuple} -> Output: {output}")
```

```
NAND GATE using Mc Culloch Pitts Neuron for any input
```

```
Enter the number of inputs for the NAND gate: 4

Testing NAND Gate with 4 inputs:
Weights: [-1 -1 -1 -1], Theta: -3
Inputs: (0, 0, 0, 0) -> Output: 1
Inputs: (0, 0, 0, 1) -> Output: 1
Inputs: (0, 0, 1, 0) -> Output: 1
Inputs: (0, 0, 1, 1) -> Output: 1
Inputs: (0, 1, 0, 0) -> Output: 1
Inputs: (0, 1, 0, 1) -> Output: 1
Inputs: (0, 1, 1, 0) -> Output: 1
Inputs: (0, 1, 1, 1) -> Output: 1
Inputs: (1, 0, 0, 0) -> Output: 1
Inputs: (1, 0, 0, 1) -> Output: 1
Inputs: (1, 0, 1, 0) -> Output: 1
Inputs: (1, 0, 1, 1) -> Output: 1
Inputs: (1, 1, 0, 0) -> Output: 1
Inputs: (1, 1, 0, 1) -> Output: 1
Inputs: (1, 1, 1, 0) -> Output: 1
Inputs: (1, 1, 1, 1) -> Output: 0
```

```
import numpy as np

def mc_pitts_not(x, weight, theta):
    z = x * weight
    return 1 if z >= theta else 0

print("NOT GATE using Mc Culloch Pitts Neuron\n")

x = int(input("Enter input (0 or 1): "))

weight = -1
theta = 0

print(f"\nWeight: {weight}, Theta: {theta}")
output = mc_pitts_not(x, weight, theta)

print(f"Input: {x} -> Output: {output}")
```

```
NOT GATE using Mc Culloch Pitts Neuron

Enter input (0 or 1): 1

Weight: -1, Theta: 0
Input: 1 -> Output: 0
```

PERCEPTRON "AND GATE" & "OR GATE" implementation

For 2 inputs

```
def perceptron_and(x1,x2):
  w1,w2 = 1,2
  b = 1.5
  z = x1*w1 + x2*w2 + b
  return 1 if z >=0 else 0

print(" AND GATE using Perceptron\n")
for x1 in [0, 1]:
    for x2 in [0, 1]:
      print({x1}, {x2},"-", perceptron_and(x1, x2))
```

```
 AND GATE using Perceptron

{0} {0} - 1
{0} {1} - 1
{1} {0} - 1
{1} {1} - 1
```

```
def perceptron_or(x1,x2):
  w1,w2 = 2,2
  b = 0.5
  z = x1*w1 + x2*w2 + b
  return 1 if z>=0 else 0

print(" OR GATE using Perceptron\n")
```

```
for x1 in [0,1]:
  for x2 in [0,2]:
    print({x1}, {x2},"-", perceptron_or(x1, x2))
```

```
 OR GATE using Perceptron

{0} {0} - 1
{0} {2} - 1
{1} {0} - 1
{1} {2} - 1
```

Perceptron Learning Algorithm

```
import numpy as np

# AND gate dataset (with bias)
X = np.array([
    [0, 0, 1],
    [0, 1, 1],
    [1, 0, 1],
    [1, 1, 1]
])

# Labels: P = 1, N = 0
y = np.array([0, 0, 0, 1])

# Initialize weights randomly
w = np.random.randn(3)

print("Initial weights:", w)

iteration = 0
converged = False

while not converged:
    iteration += 1
    print(f"\n========== Iteration {iteration} ==========")
    converged = True

    for i, (x, label) in enumerate(zip(X, y)):
        net = np.dot(w, x)
        prediction = 1 if net >= 0 else 0

        print(f"\nSample {i+1}")
        print("Input (x):", x)
        print("Net value (w·x):", net)
        print("Predicted:", prediction, "| Actual:", label)

        # x ∈ P and misclassified
        if label == 1 and net < 0:
            print("❌ Misclassified POSITIVE → Updating w = w + x")
            w = w + x
            converged = False

        # x ∈ N and misclassified
        elif label == 0 and net >= 0:
            print("❌ Misclassified NEGATIVE → Updating w = w - x")
            w = w - x
            converged = False

        else:
            print("✅ Correct classification → No update")

        print("Updated weights:", w)

    print("\nWeights at end of iteration:", w)

print("\n🎯 Training Converged")
print("Final weights:", w)
```

```
Initial weights: [ 0.29518263 -0.05055035  0.12454488]

========== Iteration 1 ==========

Sample 1
Input (x): [0 0 1]
```

```
Net value (w·x): 0.12454487976013431
Predicted: 1 | Actual: 0
❌ Misclassified NEGATIVE → Updating w = w - x
Updated weights: [ 0.29518263 -0.05055035 -0.87545512]

Sample 2
Input (x): [0 1 1]
Net value (w·x): -0.9260054652510955
Predicted: 0 | Actual: 0
✅ Correct classification → No update
Updated weights: [ 0.29518263 -0.05055035 -0.87545512]

Sample 3
Input (x): [1 0 1]
Net value (w·x): -0.5802724865498374
Predicted: 0 | Actual: 0
✅ Correct classification → No update
Updated weights: [ 0.29518263 -0.05055035 -0.87545512]

Sample 4
Input (x): [1 1 1]
Net value (w·x): -0.6308228315610671
Predicted: 0 | Actual: 1
❌ Misclassified POSITIVE → Updating w = w + x
Updated weights: [1.29518263 0.94944965 0.12454488]

Weights at end of iteration: [1.29518263 0.94944965 0.12454488]

========== Iteration 2 ==========

Sample 1
Input (x): [0 0 1]
Net value (w·x): 0.12454487976013429
Predicted: 1 | Actual: 0
❌ Misclassified NEGATIVE → Updating w = w - x
Updated weights: [ 1.29518263  0.94944965 -0.87545512]

Sample 2
Input (x): [0 1 1]
Net value (w·x): 0.0739945347489045
Predicted: 1 | Actual: 0
❌ Misclassified NEGATIVE → Updating w = w - x
Updated weights: [ 1.29518263 -0.05055035 -1.87545512]

Sample 3
Input (x): [1 0 1]
Net value (w·x): -0.5802724865498374
Predicted: 0 | Actual: 0
✅ Correct classification → No update
Updated weights: [ 1.29518263 -0.05055035 -1.87545512]

Sample 4
```